

Template / Guide

Objectives for this Phase**Activities / Work Done**

- Data Work
- Model Work
- Coding Work

Progress Logs (Chronological)

- Date and Time
- Task Performed
- Outcome
- Issues Faced
- Next Step

Results So Far :

<Which Significant results for next set of changes?>

Challenges Encountered**Next Steps / Plan****Remarks / Reflection**

09 - 22 - First Documentation

Objectives for this Phase

Activities / Work Done

- Data Work
- Model Work
- Coding Work

Progress Logs (Chronological)

- **Date and Time**
 - September 22, 2025 - around 8:00PM
- **Task Performed**
 - Code testing, debugging, and cross checking of the code with the actual system architecture.
 - Run down and side-by-side comparison of the code with the system architecture (by part.)
 - Double checking of the system architecture to code progress.
- **Outcome**
 - The group has determined what part to focus on and which parts to smoothen out so we could proceed and progress smoothly.
- **Issues Faced**
 - There is a persistent processing error.
 - Incorrect file directory structure on Part 1.4 Dataset Splitting (specifically an issue where the test directory isn't found)
- **Next Step**
 - Continue ironing out the code while adhering to the system architecture.
 - Debug any remaining errors.
 - Improve the code base.

Results So Far :

<Which Significant results for next set of changes?>

Ex : F1-score is low ...

Accuracy is still not competitive / desirable ...

Challenges Encountered

Testing input not working...

Next Steps / Plan

Complete System Archi ...

Remarks / Reflection

Consult Sir Gernale , our Thesis Adviser, in the upcoming week for possible improvements on the training process.

09 - 23 - Second Documentation

Objectives for this Phase

- To fix the previous errors, specifically the processing error at part **1.4 - Dataset Splitting**, the issue at part **2.1 - Data Generator**, the directory error at part **4 - Testing Module**, and the grad-cam implementation error at part **6 - Explainability Analysis**

Activities / Work Done

- Coding Work

Progress Logs (Chronological)

- Date and Time
 - September 23, 2025, starting from 10PM to 11:30PM
- Task Performed
 - **Part 1.4 - Dataset Splitting** - adjusted the code to properly create and organize new folders inside test dataset (rice_blast folder specifically)
 - **Part 2.1 - Data Generator** - fixed generator config to properly detect and load class folders. (healthy and rice blast)
 - **Part 4 - Testing Module** - resolved directory error
 - **Part 6 - Explainability Analysis** - fixed the grad-cam implementation error caused by incorrect tensor handling
- Outcome
 - Test pipeline fully functional.
 - Grad-CAM visualizations are successfully generated. Heatmaps appear on the last part but there is still an issue with the model and how it detects the rice-leaf blast disease.
 - PH dataset test results are successfully saved inside the output_base folder.
- Issues Faced
 - Model issue where the heatmap incorrectly shows the rice leaf blast disease.
 - Gradcam initially produced an attribute error.
 - Images were misclassified due to model sensitivity.
- Next Step

Results So Far :

- **VGG16 Baseline** was successfully evaluated and tested on images.

- Test performance metrics have been plotted for comparison purposes.
- The Grad-CAM heatmaps provide explainability insights into model predictions.

Challenges Encountered

- Dealing with the UndefinedMetricWarning during recall and precision computation.
- Need to manually check the directory structure before running the testing pipeline.
- Handling an imbalanced dataset leads to skewed prediction.

Next Steps / Plan

Change of Local Dataset for testing

<https://www.kaggle.com/datasets/gettingintoml/zambali-rice-dataset-v3-1> -> New Local dataset

- Add new testing models for VGG16 + PLSR and VGG16 + XGBoost for improved comparisons.
- Re-run the gradcam analysis for each model.

Remarks / Reflection

- Debugging process ensured that the full pipeline (train - test - explain) works. Gradcam outputs finally work but work still needs to be done to ensure the results of the model are correct.

09 - 24 - Third Documentation

Objectives for this Phase

- Use a different dataset for training since previous dataset had

Activities / Work Done

- Data Work
- Coding Work

Progress Logs (Chronological)

- September 24, 2025, starting from 1PM to 5:50PM
- Task Performed
 - Training the Unchanged models to the new dataset
- Outcome
 - Same as before, the pipeline of the architecture works with the new dataset
- Issues Faced
 - The Main Developer struggled to apply the bug fixes from the Support Coders. The Main Developer realized they didn't apply the changes thoroughly enough.
- Next Step
 - Apply the missing parts of the Pre-Processing Module to the main code

Results So Far :

<Which Significant results for next set of changes?>

Challenges Encountered

- Nothing of note besides applying the bug fixes

Next Steps / Plan

- Recode some cells in the pre-processing module since it looks like it has missing parts but it is actually coded differently than expected. Annotation and labelling is done in an excel file which is far from annotating the images itself.
- Use 50 epochs for the training.

Metric	Thesis Implemented (alpha = .125, beta = .125)	Base Model	Comparison
Loc F1 (test)	89.8468%	89.6564%	Thesis Implemented is slightly higher.
Cif F1 (test)	74.5853%	72.4542%	Thesis Implemented is notably higher.
OA (test)	96.0056%	95.9130%	Thesis Implemented is marginally higher.
mIoU (test)	67.7623%	66.4770%	Thesis Implemented is higher.
Sub class IoU (test)	80.2716%	80.4947%	Base Model is slightly higher.
Sub class F1 (no_damage)	0.89568575	0.899229	Base Model is slightly higher.
Sub class F1 (minor_damage)	0.63428496	0.59077582	Thesis Implemented is higher.
Sub class F1 (major_damage)	0.75241675	0.74889343	Thesis Implemented is slightly higher.

```
# Set paths to datasets
DATA_DIR_BANGLADESH = "E:/Codes/Jupyter/datasets/RiceLeaf/GlobalRiceLeaf/BangladeshiCrops/BangladeshiCrops/crop___Disease/Rice"
DATA_DIR_PH = "E:/Codes/Jupyter/datasets/RiceLeaf/LocalRiceLeaf/resized_raw_images/resized_raw_images"
OUTPUT_BASE = "E:/Codes/Jupyter/output/Output_Base"
```

Remarks / Reflection

- [Main Dev] I have got to read the code 5x before I ask

10 - 06 - Sixth Documentation

Objectives for this Phase

Activities / Work Done

- Coding Work

Progress Logs (Chronological)

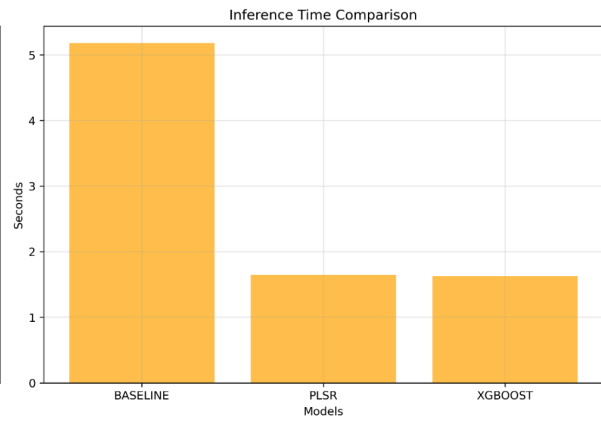
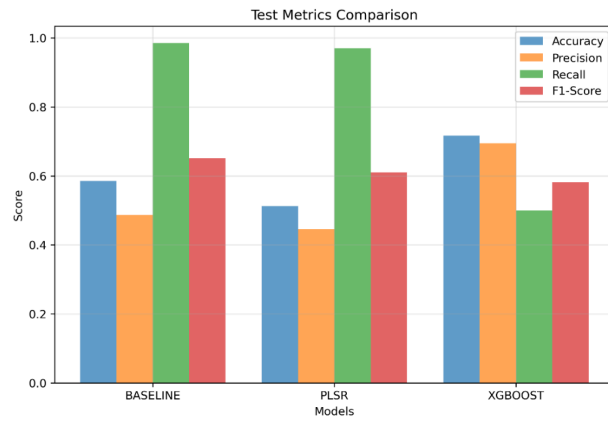
- October 06, 2025, starting from 6PM to 11:59PM
- Task Performed
 - Reorganized the code and fixed the data augmentation.
- Outcome
 - Although the data augmentation has been implemented, it was not used as the code deemed the dataset balance so it needs a bit more tinkering
- Issues Faced
 - Duplicate and Blurry detection code invalidating all the images. GradCam code not working anymore in the new quadriga system architecture
- Next Step
 - Tinker with Duplicate and Blurry detection code, data augmentation code, and the gradcam code for explainable analysis

Results So Far :

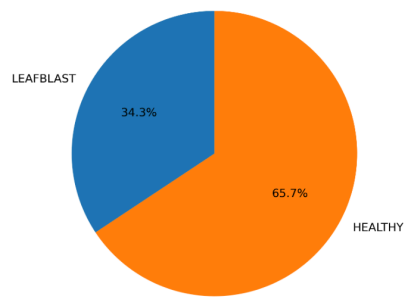
Model	Validation_Accuracy	Test_Accuracy	Test_Precision	Test_Recall	Test_F1_Score	Test_Specificity	Inference_Time_Seconds
BASELINE	0.7824	0.5855	0.4864	0.985	0.6512	0.3269	5.1783
VGG16+PLSR	0.6749	0.5128	0.4450	0.970	0.6101	0.2168	1.6465
VGG16+XGBOOST	0.7879	0.7171	0.6944	0.500	0.5814	0.8576	1.6246

VGG16+PLSR vs BASELINE: MUCH WORSE ($\Delta = -0.0727$)

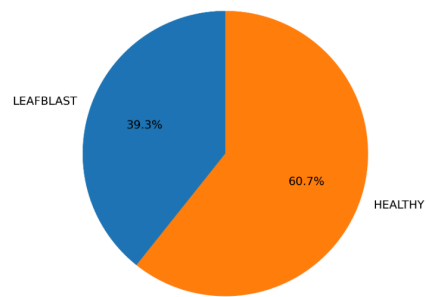
VGG16+XGBOOST vs BASELINE: MUCH BETTER ($\Delta = +0.1316$)



Training Set Distribution



Test Set Distribution



Remarks / Reflection

10 - 13 - Seventh Documentation

Objectives for this Phase

Activities / Work Done

- Data Work
- Model Work
- Coding Work

Progress Logs (Chronological)

- October 13, 2025, starting from 4PM to 11:59PM
- Task Performed
 - Preprocessing:
 - ~~Blur and Duplicate count detection~~
 - Add Image Segmentation Methods to be used: **(All on preprocessing)**
 - ~~Removing background~~
 - ~~Adaptive thresholding~~
 - ~~Double Inverse Binary Thresholding~~
 - Data Augmentation
 - ~~Lightning~~
 - Test run of Model running <- **After Training Part is done specifically yung explainability**
 - ~~Data Augmentation~~
 - ~~W/out Data Augmentation~~
 - Explainability code
- Outcome
 -
- Issues Faced
- Next Step

Results So Far :

Without Data Augmentation

Model	Validation_Accuracy	Test_Accuracy	Test_Precision	Test_Recall	Test_F1_Score	Test_Specificity	Inference_Time_Seconds
BASELINE							
VGG16+PLSR							

VGG16+ XGBOOST							
-------------------	--	--	--	--	--	--	--

VGG16+PLSR vs BASELINE: MUCH WORSE ($\Delta = -0.0727$)

VGG16+XGBOOST vs BASELINE: MUCH BETTER ($\Delta = +0.1316$)

With Data Augmentation

Model	Validation_Accuracy	Test_Accuracy	Test_Precision	Test_Recall	Test_F1_Score	Test_Specificity	Inference_Time_Seconds
BASELINE							
VGG16+ PLSR							
VGG16+ XGBOOST							

VGG16+PLSR vs BASELINE: MUCH WORSE ($\Delta = -0.0727$)

VGG16+XGBOOST vs BASELINE: MUCH BETTER ($\Delta = +0.1316$)

Challenges Encountered

Next Steps / Plan

Remarks / Reflection

10 - 19 - Eight Documentation

Objectives for this Phase

Activities / Work Done

- Data Work
- Model Work
- Coding Work

Progress Logs (Chronological)

- October 19, 2025, starting from 4PM to 11:59PM
- Task Performed
 - Preprocessing:
 - ~~Blur and Duplicate count detection~~
 - Add Image Segmentation Methods to be used: **(All on preprocessing)**
 - ~~Removing background~~
 - ~~Adaptive thresholding~~
 - ~~Double Inverse Binary Thresholding~~
 - Data Augmentation
 - ~~Lightning~~
 - Test run of Model running <- **After Training Part is done specifically yung explainability**
 - ~~Data Augmentation~~
 - ~~W/out Data Augmentation~~
 - ~~Explainability code~~
- Outcome
 -
- Issues Faced
 - Figures for bar and beeswarm saves a blank figure. So have to manually screenshot result
- Next Step

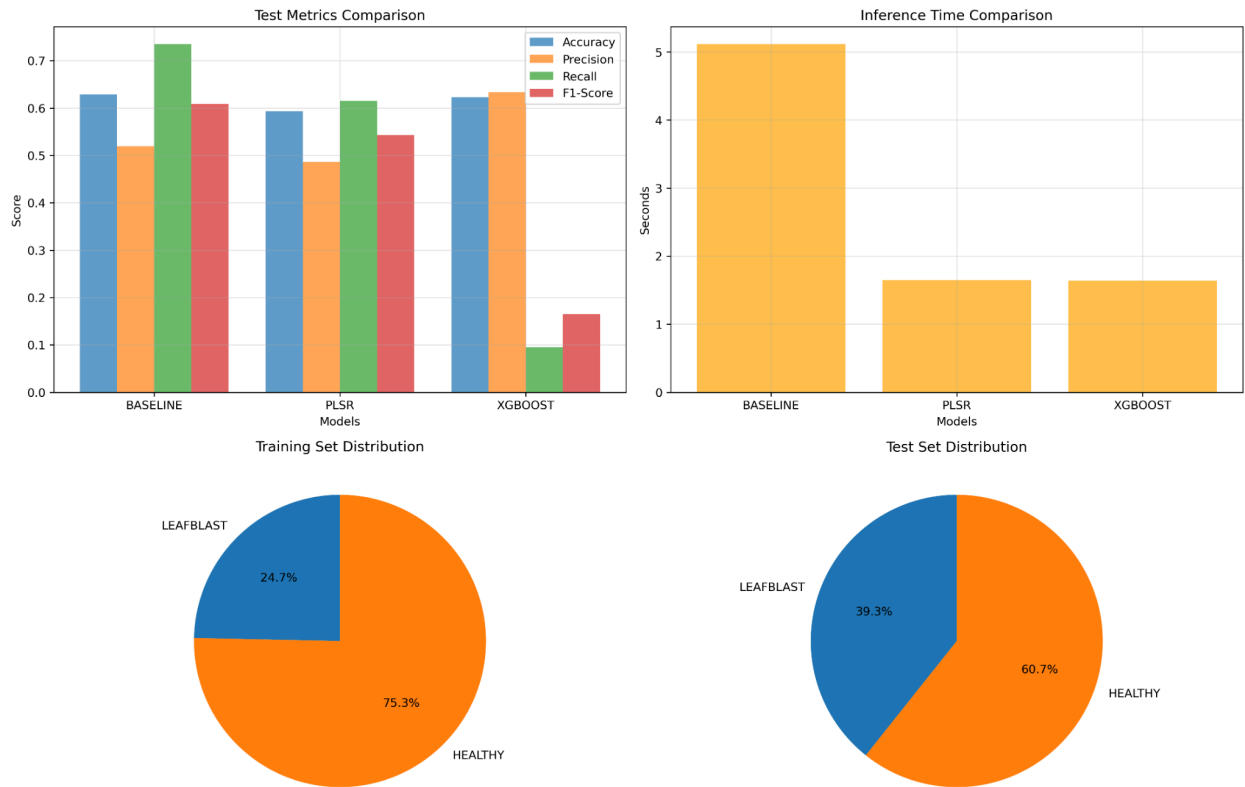
Results So Far :

Without Data Augmentation

Model	Validation_Accuracy	Test_Accuracy	Test_Precision	Test_Recall	Test_F1_Score	Test_Specificity	Inference_Time_Seconds
BASELINE	0.8506	0.6287	0.5194	0.735	0.6087	0.5599	5.1145

VGG16+ PLSR	0.7899	0.5933	0.4862	0.615	0.5430	0.5793	1.6479
VGG16+ XGBOOST	0.8253	0.6228	0.6333	0.095	0.1652	0.9644	1.6396

PLSR vs BASELINE: WORSE ($\Delta = -0.0354$)
XGBOOST vs BASELINE: NO SIGNIFICANT DIFFERENCE ($\Delta = -0.0059$)

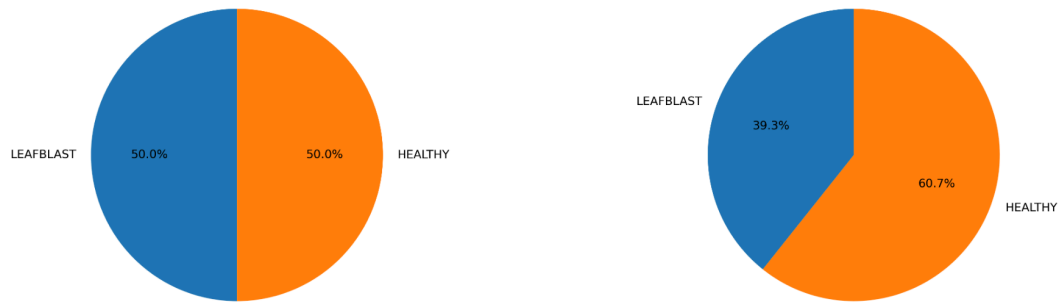
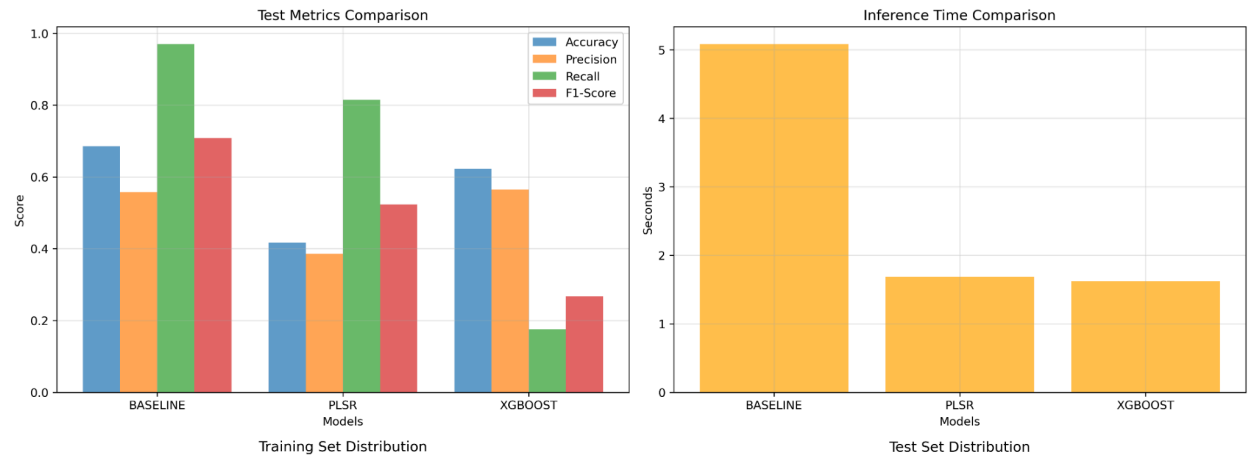


With Data Augmentation

Model	Validation_Accuracy	Test_Accuracy	Test_Precision	Test_Recall	Test_F1_Score	Test_Specificity	Inference_Time_Seconds
BASELINE	0.8380	0.6857	0.5575	0.970	0.7080	0.5016	5.0822
VGG16+ PLSR	0.6835	0.4165	0.3853	0.815	0.5233	0.1586	1.6832

VGG16+ XGBOOST	0.8329	0.6228	0.5645	0.175	0.2672	0.9126	1.6194
-------------------	--------	--------	--------	-------	--------	--------	--------

PLSR vs BASELINE: MUCH WORSE ($\Delta = -0.2692$)
XGBOOST vs BASELINE: MUCH WORSE ($\Delta = -0.0629$)



Challenges Encountered

Next Steps / Plan

Remarks / Reflection

10-30 - Ninth Documentation

Objectives for this Phase

Activities / Work Done

- Coding Work

Progress Logs (Chronological)

- October 31, 2025, starting from 4PM to 11:59PM
- Task Performed
 - Subfolder Change by choosing one of two cells at the start of the code
 - Training Models are now as discussed below:

Baseline is

```
Input Image (224x224x3)
↓
Pre-trained VGG16 (frozen weights)
↓
Global Average Pooling
↓
Dense(512) → Dropout(0.5)
↓
Dense(256) → Dropout(0.3)
↓
Dense(1) with Sigmoid → Binary Classification
```

Data Flow:

Raw images → Preprocessed images → VGG16 feature extraction → Custom classifier → Prediction

Training:

VGG16 base is frozen (pre-trained on ImageNet)

Only the custom top layers are trained on your rice leaf data

Uses binary cross-entropy loss

VGG16 + PLSR (Hybrid Model)

```
Input Image (224x224x3)
↓
Pre-trained VGG16 (frozen weights)
↓
Feature Extraction (Global Average Pooling layer)
↓
Feature Vector (512-25088 dimensions)
↓
StandardScaler (normalization)
```

↓
 PLS Regression (dimensionality reduction + classification)
 ↓
 Threshold (0.5) → Binary Classification

Data Flow:

Raw images → Preprocessed images → VGG16 feature extraction → Feature normalization → PLSR model → Prediction

Training:

VGG16 extracts features (same as baseline)

Features are normalized using StandardScaler

PLSR reduces dimensions and performs classification in one step

Uses linear regression with latent variables

VGG16 + XGBoost (Hybrid Model)

Input Image (224x224x3)
 ↓
 Pre-trained VGG16 (frozen weights)
 ↓
 Feature Extraction (Global Average Pooling layer)
 ↓
 Feature Vector (512-25088 dimensions)
 ↓
 StandardScaler (normalization)
 ↓
 XGBoost Classifier (ensemble of decision trees)
 ↓
 Binary Classification

Data Flow:

Raw images → Preprocessed images → VGG16 feature extraction → Feature normalization → XGBoost ensemble → Prediction

Training:

VGG16 extracts features (same as baseline)

Features are normalized using StandardScaler

XGBoost builds an ensemble of decision trees

Uses gradient boosting with tree-based models

- Class Balance Code
- Augmentation increases both classes based on a percentage
- Outcome
 -
- Issues Faced
 -

- Next Step

Results So Far :

Without augmentation

```
Generating comprehensive performance report...
=====
COMPREHENSIVE PERFORMANCE REPORT
=====

PERFORMANCE COMPARISON ACROSS MODELS:
=====
      Model  Val_Accuracy  Test_Accuracy  Test_Precision  Test_Recall  \
0  BASELINE      0.8253      0.5874      0.4828      0.70
1    PLSR      0.8127      0.3654      0.3660      0.84
2  XGBOOST      0.8101      0.6031      0.4737      0.09

      Test_F1_Score  Test_Specificity  Inference_Time_Seconds  Performance_Gap
0          0.5714          0.5146              4.9086          0.2379
1          0.5099          0.0583              1.5371          0.4472
2          0.1513          0.9353              1.5724          0.2070

=====
DATASET STATISTICS
=====
      Dataset  Total_Images  LEAFBLAST_Count  HEALTHY_Count  \
0  Training (Bangladesh)      779          390          389
1  Validation (Bangladesh)    195           97           98
2  Testing (Philippines)    400          200          200

      LEAFBLAST_Percentage
0          50.06
1          49.74
2          50.00

=====
MODEL COMPARISON ANALYSIS
=====
Best Performing Model: XGBOOST (Test Accuracy: 0.6031)
BASELINE vs XGBOOST: SLIGHTLY WORSE ( $\Delta = -0.0157$ )
  BASELINE Generalization: LARGE OVERFITTING (Val-Test gap: +0.2379)
PLSR vs XGBOOST: MUCH WORSE ( $\Delta = -0.2377$ )
  PLSR Generalization: LARGE OVERFITTING (Val-Test gap: +0.4472)
  XGBOOST Generalization: LARGE OVERFITTING (Val-Test gap: +0.2070)
```

With augmentation (30% added augmented data) [no reason for this specific percent jst to make it run with augmented data]

```
Generating comprehensive performance report...
=====
COMPREHENSIVE PERFORMANCE REPORT
=====

PERFORMANCE COMPARISON ACROSS MODELS:
=====
| | Model | Val_Accuracy | Test_Accuracy | Test_Precision | Test_Recall | \
0 | BASELINE | 0.8203 | 0.4931 | 0.4181 | 0.740
1 | PLSR | 0.7899 | 0.3929 | 0.3908 | 0.975
2 | XGBOOST | 0.8152 | 0.5226 | 0.3223 | 0.195
|
| Test_F1_Score | Test_Specificity | Inference_Time_Seconds | Performance_Gap
0 | 0.5343 | 0.3333 | 5.1016 | 0.3271
1 | 0.5579 | 0.0162 | 1.5342 | 0.3969
2 | 0.2430 | 0.7346 | 1.5412 | 0.2926
|
=====
DATASET STATISTICS
=====
| | | | | Dataset | Total_Images | LEAFBLAST_Count | HEALTHY_Count | \
0 | | | | | Training (Bangladesh) | 1315 | 659 | 656
1 | | | | | Validation (Bangladesh) | 195 | 97 | 98
2 | | | | | Testing (Philippines) | 400 | 200 | 200
|
| LEAFBLAST_Percentage
0 | 50.11
1 | 49.74
2 | 50.00
|
=====
MODEL COMPARISON ANALYSIS
=====
Best Performing Model: XGBOOST (Test Accuracy: 0.5226)
BASELINE vs XGBOOST: WORSE (Δ = -0.0295)
| BASELINE Generalization: LARGE OVERFITTING (Val-Test gap: +0.3271)
PLSR vs XGBOOST: MUCH WORSE (Δ = -0.1297)
| PLSR Generalization: LARGE OVERFITTING (Val-Test gap: +0.3969)
| XGBOOST Generalization: LARGE OVERFITTING (Val-Test gap: +0.2926)
```

Challenges Encountered

We've noticed that the preprocessed image made the models perform worse since the way it removes background is horrible and leaves some circle which the model interpret as being a part of the subject

Next Steps / Plan

Monday :

- **TRIPLE CHECK CODE** (esp. training and validation process) -> ken & jv help james to find holes if ever meron w/ help of chatgpt or deepseek to detect if meron dins.
- **IMPROVE PRE-PROCESSING** making sure only rice leaf blast spots are thresholded (that helps on CNN detection/feature extraction on later procedures)
- **PROTOCOL** : if more than 1/4 of the metrics are below 0.65'ish -> hab to find possible problem & do the hyperfix

If by Tuesday still cant improve METRICS 85% + and MORE THAN 1/4 of the metrics still BELOW 70% -> CONSULTATION SIR GERNALE wednesday online (if possible) + Hope Thursday Acula consultation helps.

Cuz we r kinda stuck in dis loop of trying methods to improve metrics for some time and reminding that Nov. 29 defense natin so approximately around nov. 24 expected tau na may complete na and it takes 1-2 weeks to complete documentation of chapter 4-5 so around Nov. 10 is like final internal deadline to this training that we still need to have major fix.

Remarks / Reflection

11 - 03 - Tenth Documentation

Objectives for this Phase

Activities / Work Done

- Coding Work

Progress Logs (Chronological)

- November 3, 2025, starting from 4PM to 11:59PM
- Task Performed
 - Pre-processing : remove thresholding first, go from scratched images wala muna paggawa ng black and white thingy sa bg.
 - Auto balance part : Balanced Bar graph and pie chart (like the Before of EDA)
-
- Outcome
 -
- Issues Faced
 -
- Next Step

Results So Far :

Without augmentation

```
Generating comprehensive performance report...
=====
COMPREHENSIVE PERFORMANCE REPORT
=====

PERFORMANCE COMPARISON ACROSS MODELS:
=====
      Model  Val_Accuracy  Test_Accuracy  Test_Precision  Test_Recall  \
0  BASELINE      0.7179      0.4872      0.4272      0.895
1    PLSR      0.6821      0.4283      0.3882      0.790
2  XGBOOST      0.7846      0.4951      0.4286      0.855

      Test_F1_Score  Test_Specificity  Inference_Time_Seconds  Performance_Gap
0           0.5784           0.2233              5.5916           0.2307
1           0.5206           0.1942              1.5377           0.2538
2           0.5710           0.2621              1.5632           0.2895

=====
DATASET STATISTICS
=====
      Dataset  Total_Images  LEAFBLAST_Count  HEALTHY_Count  \
0  Training (Bangladesh)      779           390           389
1  Validation (Bangladesh)    195           97           98
2   Testing (Philippines)    400          200          200

      LEAFBLAST_Percentage
0           50.06
1           49.74
2           50.00

=====
MODEL COMPARISON ANALYSIS
=====
Best Performing Model: XGBOOST (Test Accuracy: 0.4951)
BASELINE vs XGBOOST: COMPARABLE TO BEST ( $\Delta = -0.0079$ )
  BASELINE Generalization: LARGE OVERFITTING (Val-Test gap: +0.2307)
PLSR vs XGBOOST: MUCH WORSE ( $\Delta = -0.0668$ )
  PLSR Generalization: LARGE OVERFITTING (Val-Test gap: +0.2538)
  XGBOOST Generalization: LARGE OVERFITTING (Val-Test gap: +0.2895)
```

Without Augmentation: (Another Run)

Generating comprehensive performance report...

COMPREHENSIVE PERFORMANCE REPORT

PERFORMANCE COMPARISON ACROSS MODELS:

	Model	Val_Accuracy	Test_Accuracy	Test_Precision	Test_Recall	\
0	BASELINE	0.7115	0.5393	0.4689	0.9953	
1	PLSR	0.6506	0.4376	0.4154	0.9387	
2	XGBOOST	0.7019	0.5086	0.4502	0.9387	
	Test_F1_Score	Test_Specificity	Inference_Time_Seconds	Performance_Gap		
0	0.6375	0.2265	2.5308	0.1722		
1	0.5760	0.0939	1.2376	0.2130		
2	0.6086	0.2136	1.1827	0.1933		

DATASET STATISTICS

	Dataset	Total_Images	LEAFBLAST_Count	HEALTHY_Count	\
0	Training (Bangladesh)	1246	623	623	
1	Validation (Bangladesh)	312	156	156	
2	Testing (Philippines)	424	212	212	
	LEAFBLAST_Percentage				
0	50.0				
1	50.0				
2	50.0				

MODEL COMPARISON ANALYSIS

Best Performing Model: BASELINE (Test Accuracy: 0.5393)

BASELINE Generalization: LARGE OVERFITTING (Val-Test gap: +0.1722)

PLSR vs BASELINE: MUCH WORSE ($\Delta = -0.1017$)

PLSR Generalization: LARGE OVERFITTING (Val-Test gap: +0.2130)

XGBOOST vs BASELINE: WORSE ($\Delta = -0.0307$)

XGBOOST Generalization: LARGE OVERFITTING (Val-Test gap: +0.1933)

With augmentation (30% added augmented data) [no reason for this specific percent jst to make it run with augmented data]

```
Generating comprehensive performance report...
=====
COMPREHENSIVE PERFORMANCE REPORT
=====

PERFORMANCE COMPARISON ACROSS MODELS:
=====
      Model  Val_Accuracy  Test_Accuracy  Test_Precision  Test_Recall  \
0  BASELINE      0.7538      0.5226      0.4467      0.900
1    PLSR      0.7282      0.4322      0.3774      0.685
2  XGBOOST      0.7692      0.5403      0.4581      0.930

      Test_F1_Score  Test_Specificity  Inference_Time_Seconds  Performance_Gap
0          0.5970          0.2783              5.3258          0.2313
1          0.4867          0.2686              1.5743          0.2960
2          0.6139          0.2880              1.5615          0.2290

=====
DATASET STATISTICS
=====
      Dataset  Total_Images  LEAFBLAST_Count  HEALTHY_Count  \
0  Training (Bangladesh)      1012           507          505
1  Validation (Bangladesh)      195           97           98
2  Testing (Philippines)      400          200          200

      LEAFBLAST_Percentage
0          50.10
1          49.74
2          50.00

=====
MODEL COMPARISON ANALYSIS
=====
Best Performing Model: XGBOOST (Test Accuracy: 0.5403)
BASELINE vs XGBOOST: SLIGHTLY WORSE ( $\Delta = -0.0177$ )
  BASELINE Generalization: LARGE OVERFITTING (Val-Test gap: +0.2313)
PLSR vs XGBOOST: MUCH WORSE ( $\Delta = -0.1081$ )
  PLSR Generalization: LARGE OVERFITTING (Val-Test gap: +0.2960)
XGBOOST Generalization: LARGE OVERFITTING (Val-Test gap: +0.2290)
```

With augmentation: (Another Run)

```
Generating comprehensive performance report...
=====
COMPREHENSIVE PERFORMANCE REPORT
=====

PERFORMANCE COMPARISON ACROSS MODELS:
=====
  Model  Val_Accuracy  Test_Accuracy  Test_Precision  Test_Recall  \
0  BASELINE      0.7500      0.6104      0.5111      0.9764
1    PLSR      0.6859      0.4415      0.4151      0.9104
2  XGBOOST      0.7500      0.5854      0.4952      0.9764

  Test_F1_Score  Test_Specificity  Inference_Time_Seconds  Performance_Gap
0          0.6710           0.3592                2.4695          0.1396
1          0.5702           0.1197                1.2376          0.2444
2          0.6571           0.3172                1.2302          0.1646

=====
DATASET STATISTICS
=====
      Dataset  Total_Images  LEAFBLAST_Count  HEALTHY_Count  \
0  Training (Bangladesh)      1618           809           809
1  Validation (Bangladesh)      312           156           156
2   Testing (Philippines)      424           212           212

  LEAFBLAST_Percentage
0              50.0
1              50.0
2              50.0

=====
MODEL COMPARISON ANALYSIS
=====
Best Performing Model: BASELINE (Test Accuracy: 0.6104)
  BASELINE Generalization: LARGE OVERFITTING (Val-Test gap: +0.1396)
PLSR vs BASELINE: MUCH WORSE ( $\Delta$  = -0.1689)
  PLSR Generalization: LARGE OVERFITTING (Val-Test gap: +0.2444)
XGBOOST vs BASELINE: WORSE ( $\Delta$  = -0.0250)
  XGBOOST Generalization: LARGE OVERFITTING (Val-Test gap: +0.1646)
```

Challenges Encountered

Next Steps / Plan

Remarks / Reflection

Coding

From: Executed last Sunday Code
Missing Parts: Based from
System Archi

1,2,3 - Pre-processing

- 1.) Annotation Cleansing / Data labelling
↳ refer to data dictionary
- 2.) Generate CSV or JSON file of valid & invalid rice leaf images
- 3.) Dataset Augmentation (on training set only)
↳ FEASIBLE $\frac{1}{2}$ → fast GPU

Fixes:

- 1.) Testing module not working yet

To be experimented after bug fix:

- Testing (Model Deployment Module)
- Reporting Module
- Explainability (Supposedly on training)

At last after all these are done:

- Hyperparameter tuning (Training model again then test loop)

Apply Ken bug fixes to the main code

Reporting Module look like this

Metric	Thesis Implemented (alpha = .125, beta = .125)	Base Model	Comparison
Loc F1 (test)	89.8468%	89.6564%	Thesis Implemented is slightly higher.
Clf F1 (test)	74.5853%	72.4542%	Thesis Implemented is notably higher.
OA (test)	96.0056%	95.9130%	Thesis Implemented is marginally higher.
mIoU (test)	67.7623%	66.4770%	Thesis Implemented is higher.
Sub class IoU (test)	80.2716%	80.4947%	Base Model is slightly higher.
Sub class F1 (no_damage)	0.89568575	0.899229	Base Model is slightly higher.
Sub class F1 (minor_damage)	0.63428496	0.59077582	Thesis Implemented is higher.
Sub class F1 (major_damage)	0.75241675	0.74889343	Thesis Implemented is slightly higher.

From Sept. 25 Consultation Ma'am Acula :

GradCAM results were normal - Ma'am Acula

-> Because the datasets were imbalance

-> Thus, focus on the training dataset first.

-> Make dataset augmentation CODE work first

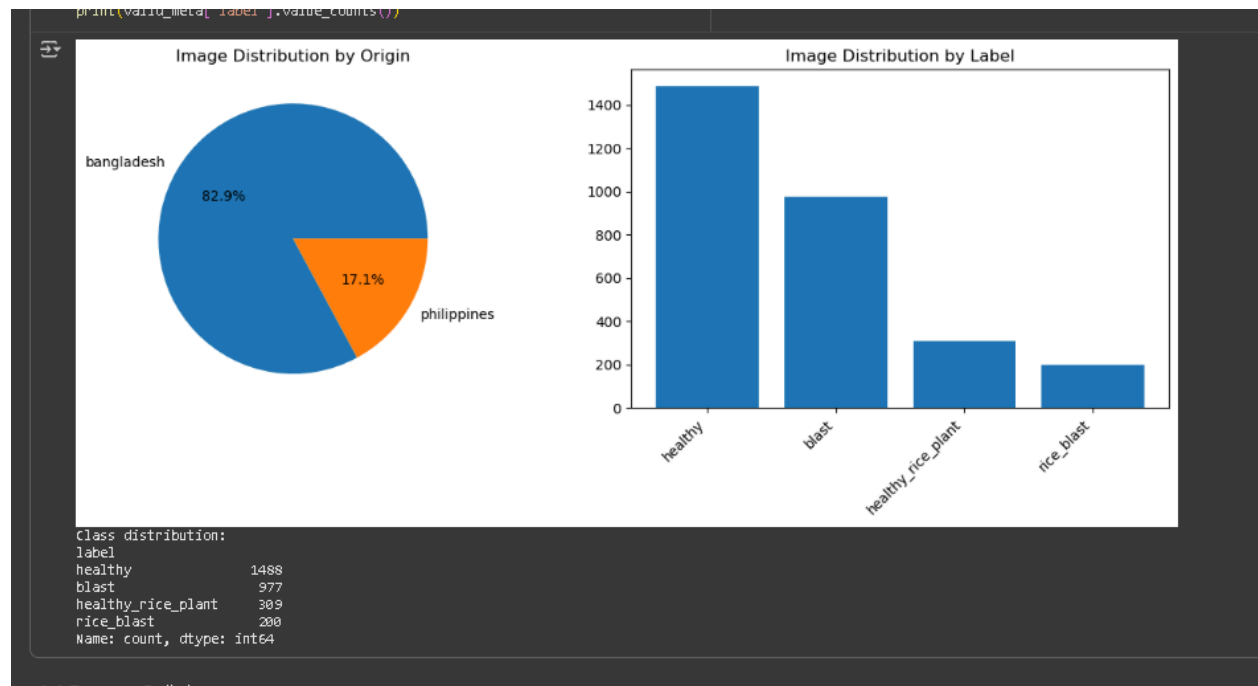
-> Also document the Dataset augmentation techniques used on the code.

Dataset hotfix :

-> reduce the healthy leaf images, make it balanced compared to the rice leaf images with rice leaf blast.

-> test the result with balanced classes (focus on training dataset only nothing else, as a part of practice)

JAMES VINCENT VALLES



Coding 11/04

James To do's

- 1.) Sa Testing Final Report -
final_comprehensive_report.png
- Add specificity

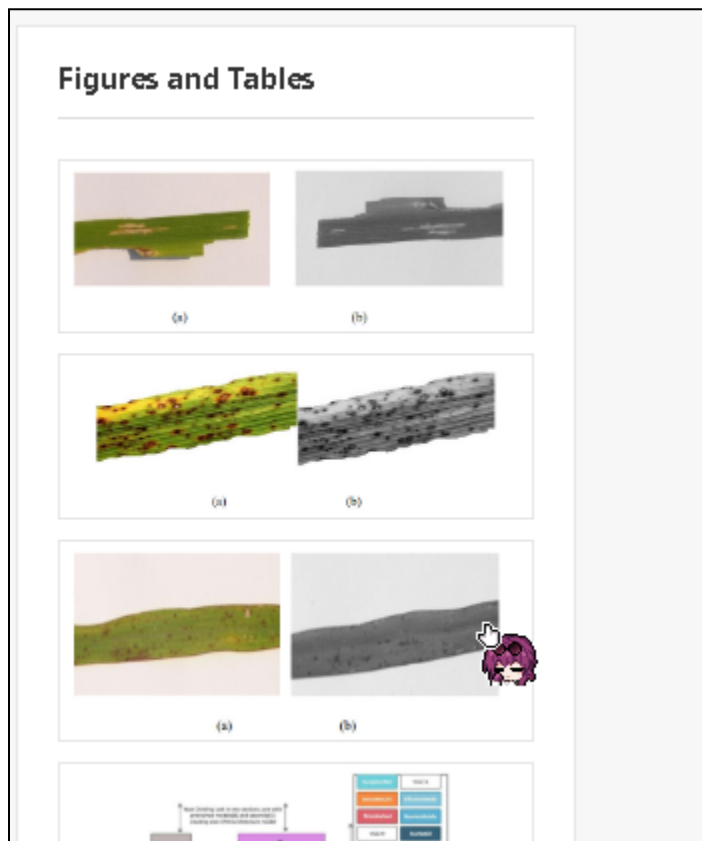


- 2.) Can we have a run without early stopping implementation. So early stopping won't happen due to possible overfitting on 50 and 100 epochs, this part is for hyperparameter tuning purposes.

```
callbacks = [  
    EarlyStopping(patience=5, restore_best_weights=True),  
    ReduceLROnPlateau(patience=3, factor=0.5)  
]
```

3.) [FOR DISCUSSION later 4PM] Add grayscale of pre-processing? (I mean literally gray'ing the image???) -> Purpose : The color of the rice leaves may be affecting the results. Will observe dataset later pa tho on (ESPECIALLY TRAINING) , validation , and testing.

I wish we could do something like grayscale? Like the image below.



For reference suggestion <pic below> :

🧠 Why VGG16 Might Focus on Leaf Shape

Pretrained models (like VGG16 trained on ImageNet) are **biased toward edge and contour features** — these are part of their early convolutional filters.

So when you feed in rice leaves:

- It sees the **outline/shape** of the leaf very clearly (especially if background is uniform or white).
- It might wrongly associate **leaf shape or orientation** with “diseased” or “healthy.”

This causes **false positives or false negatives**, especially when healthy and infected leaves have similar shapes but differ in texture or small lesions.

🌿 How to Reduce Shape Bias

Here are some practical steps that work **without needing special equipment or datasets**:

1 Convert to Grayscale

- ✓ Removes color influence.
- ✓ Forces the model to rely more on **texture** than on leaf hue.
- ✓ Already a great step you’re considering.

2 Apply Background Masking / Thresholding

If the background is already near-white, you can **mask it out** (make it uniform) using thresholding:

```
python

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
_, mask = cv2.threshold(gray, 200, 255, cv2.THRESH_BINARY)
image[mask == 255] = [255, 255, 255] # make pure white background
```

📄 Copy code

→ This makes sure the CNN doesn’t pick up on subtle background gradients.

Agenda 11/04

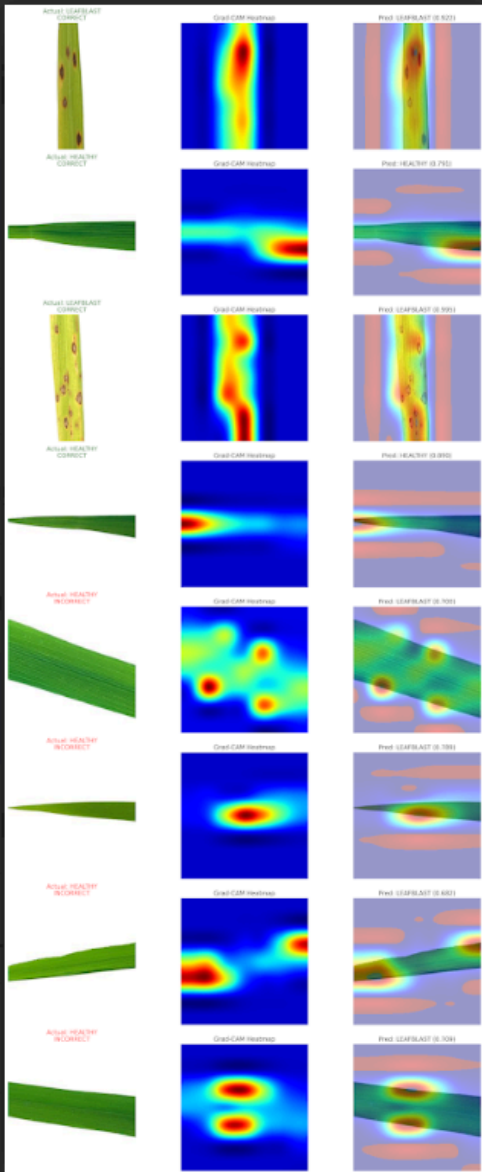
Agenda 11/04 (4PM meet soon) :

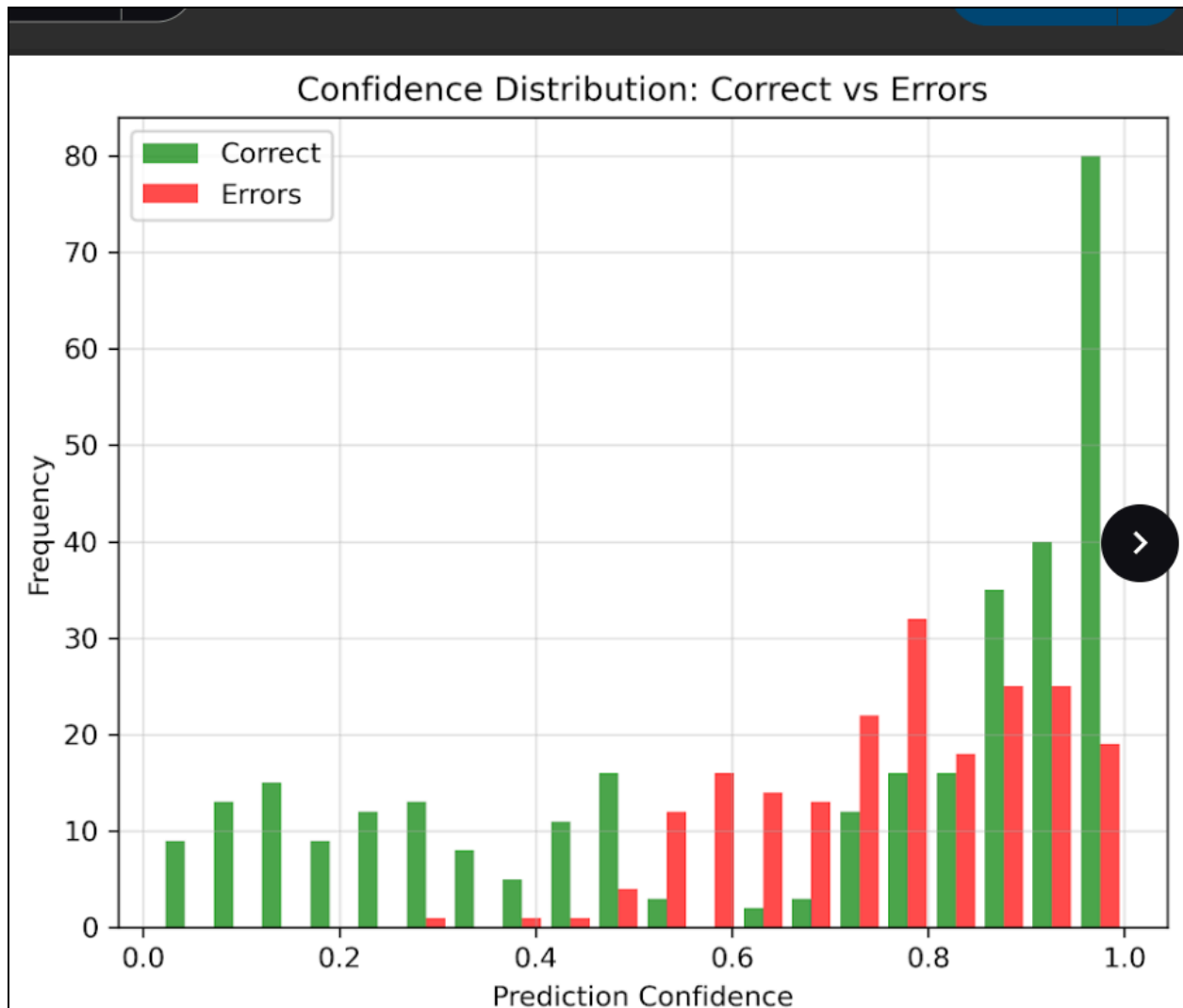
1.) Confirm RUN A and RUN B are completed first.

2.) Performance Metrics results discussion

TLDR ;

- GOOD on detecting ACTUAL rice leaf blast cases which is why we see a high percentage of recalls. [are we overpredicting infected rice leaves? , looking at confidence distribution yea it is] -> but Gradcam is showing it can actually detect the spots tho. Meanwhile, healthy rice leaves detection is not working as intended. Red spotting question mark spots.**





- **BAD at detecting ACTUAL healthy rice leaves please see specificity.**
- **The model predicted many ACTUAL healthy rice leaves as rice leaf blast, thus worsening the value result on precision and f1-score.**
- **The bad performance at detecting healthy rice leaves predicts the overall accuracy.**
- **GOAL : IMPROVE SPECIFICITY (healthy detection) and MAINTAIN RECALL (infection detection)**

- 1.) See the sources of Training (focus) , training, and validation dataset. Yung naka-balance version yea. Wanna see if color differs or other features that might off the detection.**
- 2.) Possibility of doing grayscale OR CLAHE images. Removing possibility of color and shape bias so the model can focus further on texture and spot patterns.**

CLAHE (grayscale din siya) :

What can we do to improve the model

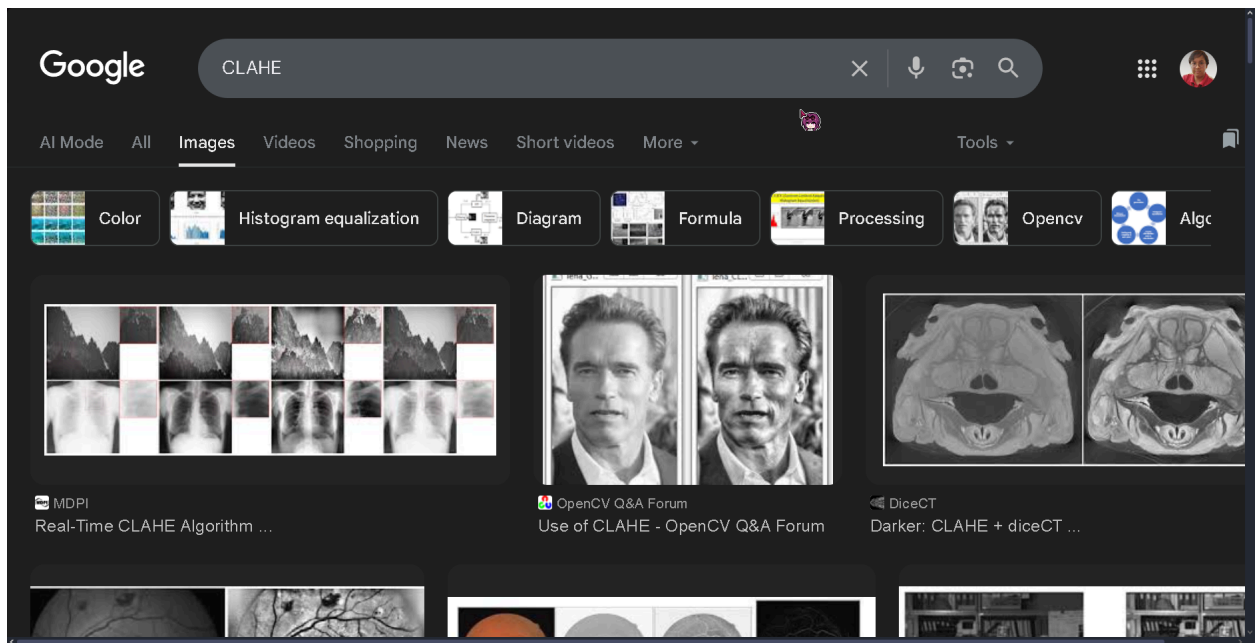
Nice — let's make this surgical and practical. Below is a prioritized, actionable plan (with quick code snippets) to improve **generalization** and **specificity** while keeping your excellent recall. I'll rank changes by likely impact so you can try highest-ROI items first.

1) Highest impact — fix domain shift & preprocessing

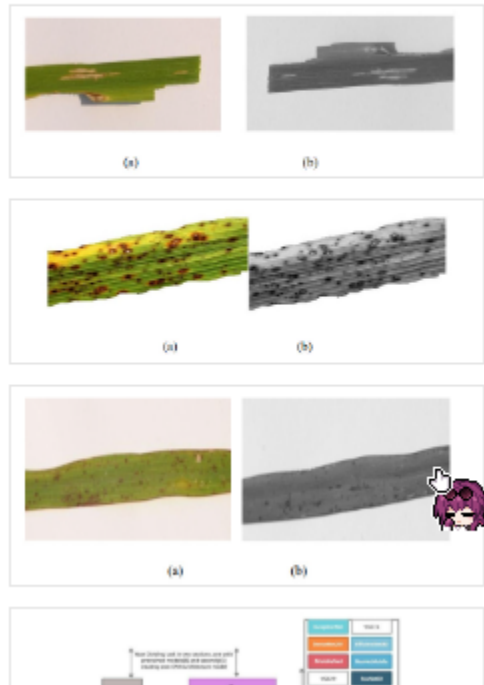
Why: Your test set is from a different country — lighting, camera, background differ. Preprocess to make domains similar.

- **Color constancy** + CLAHE + LAB (you already have code)
- **Leaf segmentation** (remove soil/background) — even heuristic HSV masks help a lot.
- **Histogram matching** between train and test (match color distribution of Bangladesh → Philippines).
- **Standardize acquisition** if possible (same cropping/resolution).

Expected: big gain in specificity and test accuracy.



Figures and Tables



🌱 How to Reduce Shape Bias

Here are some practical steps that work **without needing special equipment or datasets**:

1 Convert to Grayscale

- ✓ Removes color influence.
- ✓ Forces the model to rely more on **texture** than on leaf hue.
- ✓ Already a great step you're considering.

2 Apply Background Masking / Thresholding

If the background is already near-white, you can **mask it out** (make it uniform) using thresholding:

python

Copy code

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
_, mask = cv2.threshold(gray, 200, 255, cv2.THRESH_BINARY)
image[mask == 255] = [255, 255, 255] # make pure white background
```

→ This makes sure the CNN doesn't pick up on subtle background gradients.

11/06 Consult Acula & Gernale

11/06 - Ma'am Acula morning consultation (TC) + Gernale Afternoon (TA)

MAJOR :

- 1.) grayscale + CLAHE -> impactful to positive healthy cases, chapters 1-4

Parts where revisions are needed to be made :

CHAPTER 1

- Conceptual Framework
- Definition of Terms

CHAPTER 2

- Image Segmentation -> Include Grayscale and CLAHE (somewhere Section B)
- (Preferred) if we can add Grayscale or CLAHE study on synthesis table

CHAPTER 3

- Research Method modify
- Research Design

CHAPTER 4

- Update complete system architecture diagram & paragraph description
- Update pre-processing diagram & paragraph description

-
- Find more things we can tweak out other than learning rate (we can ask Gernale later for suggestions)
 - We can tweak XGBoost parameters too not just VGG16 parameters. PLSR not sure if theres smtn to tweak

GUIDE TO HYPERPARAMETER TUNING :

Research XGBoost parameters to tweak:

- We can **add reg_lambda, subsample, colsample_bytree**

- We can tweak `max_depth`, `learning_rate`, `n_estimators`

5. Quick Tuning Suggestions

For immediate improvement, try these combinations:

python

 Copy

 Download

```
# Conservative settings (good starting point)
conservative_params = {
    'max_depth': 4,
    'learning_rate': 0.05,
    'n_estimators': 300,
    'subsample': 0.8,
    'colsample_bytree': 0.8,
    'reg_lambda': 2
}
```

Key Recommendations:

1. **Start with** `max_depth=4-6` - VGG features are rich, so shallow trees often work well
2. **Use lower** `learning_rate` (**0.01-0.1**) with more `n_estimators` for stability
3. **Apply regularization** (`reg_lambda=1-5`) to prevent overfitting on VGG features
4. **Tune** `subsample` and `colsample_bytree` for additional regularization
5. **Monitor with early stopping** to find optimal number of trees automatically

The most impactful parameters are typically `max_depth`, `learning_rate`, and regularization terms when working with pre-trained CNN features.

Suggestions ni Deepseek @ Ken and JV na hyperparameter tuning sample

2. Quick Parameter Combinations to Try

Conservative Approach (Less overfitting):

python

 Copy  Download

```
xgb_model = xgb.XGBClassifier(  
    n_estimators=200,      # More trees with lower learning rate  
    max_depth=4,          # Shallower trees  
    learning_rate=0.05,    # Lower learning rate  
    min_child_weight=3,    # More conservative  
    subsample=0.8,        # Random row sampling  
    colsample_bytree=0.8,  # Random feature sampling  
    reg_lambda=2,         # Stronger L2 regularization  
    objective='binary:logistic',  
    random_state=42,  
    eval_metric='logloss'  
)
```

For PLSR :

- N_components -> pwede i-tweak between 2-5

```
start_time = time.time()  
  
plsr_model = PLSRegression(n_components=2)  
plsr_model.fit(train_features_scaled, y_train)
```

2. Most impactful parameters:

- `n_components` : Controls complexity (2-5 usually good for VGG features)

Threshold - probably no need modify I think? 0.5 is fine for default
Scale - ofc we scale so always true.

Quick Start Recommendations:

1. Start with these combinations:

python

 Copy  Download

```
# Quick test combinations
test_params = [
    {'n_components': 2, 'threshold': 0.5},
    {'n_components': 3, 'threshold': 0.5},
    {'n_components': 2, 'threshold': 0.4},
    {'n_components': 4, 'threshold': 0.5}
]
```

2. Most impactful parameters:

- `n_components` : Controls complexity (2-5 usually good for VGG features)
- `threshold` : Balances precision vs recall
- `scale` : Usually better to keep as `True` for VGG features

For VGG16 baseline looking to tweak :

- We can **tweak number of neurons, dropout rate, learning rate**
- Possibly we can tweak also so we can possibly improve PLSR and XGBoost?

Number of Neurons combinations :

- (256, 128), # Smaller - less overfitting
- (512, 256), # Your current (keep this)
- (1024, 512), # Larger - more power

Dropout rate options :

- (0.3, 0.2), # Less dropout - more learning
- (0.5, 0.3), # Your current
- (0.7, 0.5), # More dropout - less overfitting

Learning Rate options :

- 0.00001, 0.0001, 0.0005

Possible combinations to test:

- (preferred for small dataset) ->(256, 128) number of neurons, (0.7, 0.5) increased dropout rate, (0.0001) learning rate
- Kau na on other options watchu think will improve, hab this as guide.

Suggested combo #2 ni deepseek :

Even Simpler - Just Change These Numbers:

python

Copy

Download

```
# CHANGE FROM:
# layers.Dense(512, activation='relu')
# layers.Dropout(0.5)
# layers.Dense(256, activation='relu')
# layers.Dropout(0.3)

# CHANGE TO:
layers.Dense(256, activation='relu') # ⬇️ Smaller
layers.Dropout(0.6)                 # ⬆️ More dropout
layers.Dense(128, activation='relu') # ⬇️ Smaller
layers.Dropout(0.4)                 # ⬆️ More dropout
```

When to Use What:

- **Small dataset** → Use smaller neurons + more dropout
- **Large dataset** → Use larger neurons + less dropout
- **Model overfitting** → Use more dropout
- **Model underfitting** → Use larger neurons

Bottom line: Try 2-3 different combinations and see which gives better accuracy. It's worth 10-15 minutes of testing!


```
base_model.trainable = False

# Add custom top layers
model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(1, activation='sigmoid') # Binary classification
])
```

dropout rates (prevents overfitting)
number of neurons

1. Number of Neurons (Most Important)

python

Copy

Download

```
# Try these combinations instead of (512, 256):
neuron_options = [
    (256, 128),    # Smaller - less overfitting
    (512, 256),    # Your current (keep this)
    (1024, 512),   # Larger - more power
]
```

2. Dropout Rates (Prevents Overfitting)

python

Copy

Download

```
# Try these instead of (0.5, 0.3):
dropout_options = [
    (0.3, 0.2),    # Less dropout - more learning
    (0.5, 0.3),    # Your current
    (0.7, 0.5),    # More dropout - less overfitting
]
```

3. Learning Rate (Training Speed)

python

Copy

Download

```
# Try these instead of 0.0001:
learning_rates = [0.00001, 0.0001, 0.0005]
```

Simple Testing Code:

ENDGOAL :

- The goal is 90 % +++ we will see with these optimizations on hyperparameter tuning.

- Models were non-significant different almost (we show the results from the 75 epoch LR=0.0001 run)

Anyways for future reference :

- > checking improvement on w/ data augment runs
- > View improvements or stagnant improvements on hyperparameter tuning. On XGBoost, PLSR, and possibly even VGG16 layers itself.

- yea our current optimizer is adam ,

-> Okay no changes here

- concern : do we add more pre-processing steps (most likely not na besides grayscale + CLAHE), so we focus again on finding ways to improve on hyperparameter tuning (we can also ChatGPT deepseek for suggestions_

-> yea doing major updates on CH 1-4 and code-wise on training is done already.

Gernale parts that not yet mentioned by Acula :

- Make.pkl file also included on being generated

-> okay

For James :

- fix data augmentation code runs (fix 1s training lmao, probably smtn wrong w/ the input?)

- > Cause : Only augmented data is being used when running data augment section

-> 3 training , 1 validation , 1 deployment pickel file (Ask Sir Gernale purpose of these

-> most likely fixed today

Timeline :

Nov. 8 (Sat) - Data Augmentation Hotfix okay na kay James  &.pkl file 

Nov. 9 (Sun) to Nov. Nov. 12 (Wed) - Hyperparameter Tuning phase

Nov. 13 (Thu) - Ask for Consultation (Acula TC on Thesis subject & Gernale TA gspace ask for F2F consult Thursday)

Nov. 14 (Fri) to Nov. 19 (Wed) - Revisions on CH 1-3 + CH 4-5 make documentation ultimatum

Nov. 20 (Thu) - initial documentation Consultation

Nov. 21 (Fri) to Nov. 27 (Thu) - Final Documentation

Nov. 27 (Thu) & Nov. 28 (Fri) - Hardcopy & Requirements hard check

Nov. 29 (Sat) - 7:00 AM Thesis II defense w/ Sir Decamora, Ma'am Ponay, and Sir Castilla.

-> Picture with Ma'am Acula Imao and other panelists (bati version)

Copy of 11/06 Consult Acula & Gernale

-
- We can tweak XGBoost parameters too not just VGG16 parameters.
- PLSR not sure if theres smtn to tweak

GUIDE TO HYPERPARAMETER TUNING :

Research XGBoost parameters to tweak:

- We can **add reg_lambda, subsample, colsample_bytree**
- We can **tweak max_depth, learning rate, n_estimators**

5. Quick Tuning Suggestions

For immediate improvement, try these combinations:

python

 Copy

 Download

```
# Conservative settings (good starting point)
conservative_params = {
    'max_depth': 4,
    'learning_rate': 0.05,
    'n_estimators': 300,
    'subsample': 0.8,
    'colsample_bytree': 0.8,
    'reg_lambda': 2
}
```

Key Recommendations:

1. **Start with** `max_depth=4-6` - VGG features are rich, so shallow trees often work well
2. **Use lower** `learning_rate (0.01-0.1)` with more `n_estimators` for stability
3. **Apply regularization** (`reg_lambda=1-5`) to prevent overfitting on VGG features
4. **Tune** `subsample` and `colsample_bytree` for additional regularization
5. **Monitor with early stopping** to find optimal number of trees automatically

The most impactful parameters are typically `max_depth` , `learning_rate` , and regularization terms when working with pre-trained CNN features.

Suggestions ni Deepseek @ Ken and JV na hyperparameter tuning sample

2. Quick Parameter Combinations to Try

Conservative Approach (Less overfitting):

```
python Copy Download  
  
xgb_model = xgb.XGBClassifier(  
    n_estimators=200,      # More trees with lower learning rate  
    max_depth=4,          # Shallower trees  
    learning_rate=0.05,   # Lower learning rate  
    min_child_weight=3,   # More conservative  
    subsample=0.8,       # Random row sampling  
    colsample_bytree=0.8, # Random feature sampling  
    reg_lambda=2,        # Stronger L2 regularization  
    objective='binary:logistic',  
    random_state=42,  
    eval_metric='logloss'  
)
```

For PLSR :

- N_components -> pwede i-tweak between 2-5

```
start_time = time.time()  
  
plsr_model = PLSRegression(n_components=2)  
plsr_model.fit(train_features_scaled, y_train)
```

2. Most impactful parameters:

- `n_components` : Controls complexity (2-5 usually good for VGG features)

Threshold - probably no need modify I think? 0.5 is fine for default

Scale - ofc we scale so always true.

Quick Start Recommendations:

1. Start with these combinations:

python

 Copy  Download

```
# Quick test combinations
test_params = [
    {'n_components': 2, 'threshold': 0.5},
    {'n_components': 3, 'threshold': 0.5},
    {'n_components': 2, 'threshold': 0.4},
    {'n_components': 4, 'threshold': 0.5}
]
```

2. Most impactful parameters:

- `n_components` : Controls complexity (2-5 usually good for VGG features)
- `threshold` : Balances precision vs recall
- `scale` : Usually better to keep as `True` for VGG features

For VGG16 baseline looking to tweak :

- We can **tweak number of neurons, dropout rate, learning rate**
- Possibly we can tweak also so we can possibly improve PLSR and XGBoost?

Number of Neurons combinations :

- (256, 128), # Smaller - less overfitting
- (512, 256), # Your current (keep this)
- (1024, 512), # Larger - more power

Dropout rate options :

- (0.3, 0.2), # Less dropout - more learning
- (0.5, 0.3), # Your current
- (0.7, 0.5), # More dropout - less overfitting

Learning Rate options :

- 0.00001, 0.0001, 0.0005

Possible combinations to test:

- (preferred for small dataset) ->(256, 128) number of neurons, (0.7, 0.5) increased dropout rate, (0.0001) learning rate
- Kau na on other options watchu think will improve, hab this as guide.

Suggested combo #2 ni deepseek :

Even Simpler - Just Change These Numbers:

python

Copy

Download

```
# CHANGE FROM:
# layers.Dense(512, activation='relu')
# layers.Dropout(0.5)
# layers.Dense(256, activation='relu')
# layers.Dropout(0.3)

# CHANGE TO:
layers.Dense(256, activation='relu') # ⬇️ Smaller
layers.Dropout(0.6)                 # ⬆️ More dropout
layers.Dense(128, activation='relu') # ⬇️ Smaller
layers.Dropout(0.4)                 # ⬆️ More dropout
```

When to Use What:

- **Small dataset** → Use smaller neurons + more dropout
- **Large dataset** → Use larger neurons + less dropout
- **Model overfitting** → Use more dropout
- **Model underfitting** → Use larger neurons

Bottom line: Try 2-3 different combinations and see which gives better accuracy. It's worth 10-15 minutes of testing!

```
base_model.trainable = False

# Add custom top layers
model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(1, activation='sigmoid') # Binary classification
])
```

dropout rates (prevents overfitting)
number of neurons

1. Number of Neurons (Most Important)

python

 Copy

 Download

```
# Try these combinations instead of (512, 256):
neuron_options = [
    (256, 128),    # Smaller - less overfitting
    (512, 256),    # Your current (keep this)
    (1024, 512),   # Larger - more power
]
```

2. Dropout Rates (Prevents Overfitting)

python

 Copy

 Download

```
# Try these instead of (0.5, 0.3):
dropout_options = [
    (0.3, 0.2),    # Less dropout - more learning
    (0.5, 0.3),    # Your current
    (0.7, 0.5),    # More dropout - less overfitting
]
```

3. Learning Rate (Training Speed)

python

 Copy

 Download

```
# Try these instead of 0.0001:
learning_rates = [0.00001, 0.0001, 0.0005]
```

Simple Testing Code:

ENDGOAL :

- The goal is 90 % +++ we will see with these optimizations on hyperparameter tuning.

- Models were non-significant different almost (we show the results from the 75 epoch LR=0.0001 run)

Anyways for future reference :

- > checking improvement on w/ data augment runs
- > View improvements or stagnant improvements on hyperparameter tuning. On XGBoost, PLSR, and possibly even VGG16 layers itself.

Code Clipboard

Option 1: Simplify the Model

```
def create_baseline_vgg16():
    """Create a simpler VGG16 model to prevent overfitting"""
    base_model = VGG16(weights='imagenet', include_top=False,
                        input_shape=(224, 224, 3))
    base_model.trainable = False

    model = models.Sequential([
        base_model,
        layers.GlobalAveragePooling2D(),
        layers.Dense(128, activation='relu'), # Reduced from 512
        layers.Dropout(0.5),
        layers.Dense(1, activation='sigmoid')
    ])

    model.compile(
        optimizer=optimizers.Adam(learning_rate=0.0001), # Lower learning rate
        loss='binary_crossentropy',
        metrics=['accuracy']
    )

    return model, base_model
```

Option 2: Add More Regularization

```
def create_regularized_vgg16():
    """VGG16 with stronger regularization"""
    base_model = VGG16(weights='imagenet', include_top=False,
                        input_shape=(224, 224, 3))
    base_model.trainable = False

    model = models.Sequential([
        base_model,
        layers.GlobalAveragePooling2D(),
        layers.Dense(512, activation='relu', kernel_regularizer='l2'),
        layers.Dropout(0.7), # Increased dropout
        layers.Dense(256, activation='relu', kernel_regularizer='l2'),
        layers.Dropout(0.5),
        layers.Dense(1, activation='sigmoid')
    ])

    model.compile(
        optimizer=optimizers.Adam(learning_rate=0.0001),
        loss='binary_crossentropy',
        metrics=['accuracy']
    )
```

```
)
```

```
return model
```

Option 3: Check Your Data

```
# Check dataset sizes and balance
```

```
print(f"Training samples: {len(X_train)}")
```

```
print(f"Validation samples: {len(X_val)}")
```

```
print(f"Training class distribution: {np.unique(y_train, return_counts=True)}")
```

```
print(f"Validation class distribution: {np.unique(y_val, return_counts=True)}")
```

```
# Check if data is shuffled properly
```

```
print(f"First 10 training labels: {y_train[:10]}")
```

```
print(f"First 10 validation labels: {y_val[:10]}")
```

Recommendation: Try Option 1 (simpler model) first, and run the diagnostic check to see if there are data issues. The current model is definitely overfitting and won't generalize well to your Philippines test set!

Fix 1: Reduce Model Complexity

def create_simple_model():

"""Much simpler model to prevent overfitting"""

```
    model = models.Sequential([
        layers.Conv2D(32, (3, 3), activation='relu', input_shape=(224, 224, 3)),
        layers.MaxPooling2D(2, 2),
        layers.Conv2D(64, (3, 3), activation='relu'),
        layers.MaxPooling2D(2, 2),
        layers.Conv2D(128, (3, 3), activation='relu'),
        layers.MaxPooling2D(2, 2),
        layers.GlobalAveragePooling2D(),
        layers.Dropout(0.5),
        layers.Dense(128, activation='relu'),
        layers.Dropout(0.3),
        layers.Dense(1, activation='sigmoid')
    ])
```

```
    model.compile(
        optimizer='adam',
        loss='binary_crossentropy',
        metrics=['accuracy']
    )
```

return model

The Fix

2.x Data Preparation Cell - WITH AUGMENTATION SUPPORT

```
def prepare_training_data_with_augmentation(train_df, val_df, test_df):
    """Prepare training, validation, and test datasets WITH AUGMENTATION SUPPORT"""
    print("Loading and preprocessing images WITH AUGMENTATION SUPPORT...")

    # Prepare training data (may include augmented images)
    X_train = []
    y_train = []

    for _, row in train_df.iterrows():
        if row.get('is_augmented', False) and 'augmented_image' in row:
            # Use the pre-computed augmented image
            aug_img = row['augmented_image']
            if aug_img is not None:
                X_train.append(aug_img)
                y_train.append(1 if row['label'] == 'LEAFBLAST' else 0)
        else:
            # Load from original path
            img_array = load_and_preprocess_image(row['original_path'])
            if img_array is not None:
                X_train.append(img_array)
                y_train.append(1 if row['label'] == 'LEAFBLAST' else 0)

    X_train = np.array(X_train)
    y_train = np.array(y_train)

    # Prepare validation data (only original images)
    X_val = []
    y_val = []

    for _, row in val_df.iterrows():
        img_array = load_and_preprocess_image(row['original_path'])
        if img_array is not None:
            X_val.append(img_array)
            y_val.append(1 if row['label'] == 'LEAFBLAST' else 0)

    X_val = np.array(X_val)
    y_val = np.array(y_val)

    # Prepare test data (only original images)
    X_test = []
    y_test = []
```

```

for _, row in test_df.iterrows():
    img_array = load_and_preprocess_image(row['original_path'])
    if img_array is not None:
        X_test.append(img_array)
        y_test.append(1 if row['label'] == 'LEAFBLAST' else 0)

X_test = np.array(X_test)
y_test = np.array(y_test)

print(f"\nFinal dataset shapes WITH AUGMENTATION:")
print(f"Training data: {X_train.shape}, {y_train.shape}")
print(f"Validation data: {X_val.shape}, {y_val.shape}")
print(f"Test data: {X_test.shape}, {y_test.shape}")

# Verify augmentation worked
original_train_count = len([row for _, row in train_df.iterrows() if not row.get('is_augmented',
False)])
augmented_count = len([row for _, row in train_df.iterrows() if row.get('is_augmented',
False)])

print(f"\nAugmentation Verification:")
print(f"Original training samples: {original_train_count}")
print(f"Augmented samples: {augmented_count}")
print(f"Total training samples: {len(X_train)}")

return X_train, X_val, X_test, y_train, y_val, y_test

# Prepare the data WITH AUGMENTATION SUPPORT
X_train, X_val, X_test, y_train, y_val, y_test = prepare_training_data_with_augmentation(
    train_df, val_df, test_df
)

print("Data preparation with augmentation completed!")
print("="*60)
print(f"Training data shape: {X_train.shape}")
print(f"Validation data shape: {X_val.shape}")
print(f"Number of training samples: {len(X_train)}")
print(f"Number of validation samples: {len(X_val)}")
print(f"Batch size: {BATCH_SIZE}")
print(f"Steps per epoch: {len(X_train) // BATCH_SIZE}")

```

Alternative Simpler Fix

```

def prepare_training_data_simple(train_df, val_df, test_df):

```

```

"""Simple preparation that uses the augmented images from train_df"""
print("Loading training data from augmented DataFrame...")

# Training data - use augmented images directly
X_train = []
y_train = []

for _, row in train_df.iterrows():
    if 'augmented_image' in row and row['augmented_image'] is not None:
        # Use the augmented image array directly
        X_train.append(row['augmented_image'])
    else:
        # Load original image
        img_array = load_and_preprocess_image(row['original_path'])
        if img_array is not None:
            X_train.append(img_array)

    y_train.append(1 if row['label'] == 'LEAFBLAST' else 0)

X_train = np.array(X_train)
y_train = np.array(y_train)

# Validation and test data (load from paths)
X_val = [load_and_preprocess_image(row['original_path']) for _, row in val_df.iterrows()]
y_val = [1 if row['label'] == 'LEAFBLAST' else 0 for _, row in val_df.iterrows()]

X_test = [load_and_preprocess_image(row['original_path']) for _, row in test_df.iterrows()]
y_test = [1 if row['label'] == 'LEAFBLAST' else 0 for _, row in test_df.iterrows()]

# Remove None values and convert to arrays
X_val = np.array([x for x in X_val if x is not None])
y_val = np.array([y for i, y in enumerate(y_val) if X_val[i] is not None])

X_test = np.array([x for x in X_test if x is not None])
y_test = np.array([y for i, y in enumerate(y_test) if X_test[i] is not None])

print(f"\nFinal dataset shapes:")
print(f"Training data: {X_train.shape}, {y_train.shape}")
print(f"Validation data: {X_val.shape}, {y_val.shape}")
print(f"Test data: {X_test.shape}, {y_test.shape}")

return X_train, X_val, X_test, y_train, y_val, y_test

```

Tab 16

